Waiting for a Pattern

N-20261107

Toss a fair coin until three consecutive heads appear. The expected number of tosses is 14. If we wait for HTH instead, the answer is 10; for HHT, it is 8. All three words have length three, and each has probability 1/8 of appearing in any specified block of three tosses. Evidently, knowing the probability of a single block is not enough to predict how long we must wait.

The discrepancy is already present in ordinary string matching. A partial match need not be forgotten after a mismatch, and a completed word may overlap another copy of itself. We will use that observation to turn a small probability puzzle into a finite automaton, then into a Markov chain and a generating function. The same calculation will eventually reveal the entire waiting-time distribution through its tail probabilities.[1]

§1.1 Three words, three waiting times

Let $X_1, X_2, \ldots$ be independent and uniformly distributed on $\{\mathrm{H}, \mathrm{T}\}$. For a fixed word $w = w_1 \cdots w_m$, define

$$T_w = \min\{n \geq m : X_{n-m+1} \cdots X_n = w_1 \cdots w_m\}.$$

Thus T_w counts tosses up to and including the first completed occurrence of w. We count *consecutive* patterns: HHT is absent from HTHT.

A tempting calculation is

$$\Pr(X_{n-2}X_{n-1}X_n = \mathrm{HHH}) = \frac{1}{8}, \qquad \text{so perhaps } \mathbb{E}T_{\mathrm{HHH}} = 8.$$

The guess overlooks two facts. Windows ending at successive positions use many of the same tosses, and the first two tosses do not yet form a candidate block. Dividing a sequence into disjoint blocks of length three gives independent trials, but discards patterns that straddle a block boundary.

We can find the correct expectation by remembering just how much of the target has matched at the *end* of the sequence seen so far.

For HHH, there are three unfinished states:

$$0 = \text{no useful suffix}, \qquad 1 = \mathrm{H}, \qquad 2 = \mathrm{HH}.$$

Let e_0, e_1, e_2 be the expected additional tosses before success, starting in these states. Conditioning on the next toss gives

$$\begin{aligned} e_0 &= 1 + \tfrac{1}{2}e_1 + \tfrac{1}{2}e_0, \\ e_1 &= 1 + \tfrac{1}{2}e_2 + \tfrac{1}{2}e_0, \\ e_2 &= 1 + \tfrac{1}{2}\cdot 0 + \tfrac{1}{2}e_0. \end{aligned}$$

[1] The underlying theory of overlapping words is developed in Leonidas Guibas and Andrew Odlyzko, *String Overlaps, Pattern Matching, and Nontransitive Games* (1981). For the prefix machinery see Knuth, Morris and Pratt, *Fast Pattern Matching in Strings* (1977). Our calculations below are self-contained.

The 1 in each equation accounts for the toss we are about to make; the 0 in the last line reflects success if the next toss is H. The first equation says $e_0 = 2 + e_1$, and the last says $e_2 = 1 + e_0/2$. Substituting into the middle equation gives

$$e_1 = \frac{3}{2} + \frac{3}{4}e_0, \qquad e_0 = \frac{7}{2} + \frac{3}{4}e_0, \qquad \boxed{\mathbb{E}T_{\mathrm{HHH}} = 14}.$$

For HTH, the unfinished states are $0, \mathrm{H}, \mathrm{HT}$. After reading H, another head still leaves the useful suffix H; a tail advances the match to HT. From HT, a head completes the pattern and a tail leaves no useful suffix. The equations become

$$\begin{aligned} u_0 &= 1 + \tfrac{1}{2}u_1 + \tfrac{1}{2}u_0, \\ u_1 &= 1 + \tfrac{1}{2}u_1 + \tfrac{1}{2}u_2, \\ u_2 &= 1 + \tfrac{1}{2}\cdot 0 + \tfrac{1}{2}u_0. \end{aligned}$$

Now $u_0 = 2 + u_1$, $u_1 = 2 + u_2$ and $u_2 = 1 + u_0/2$, so

$$u_0 = 5 + \frac{u_0}{2}, \qquad \boxed{\mathbb{E}T_{\mathrm{HTH}} = 10}.$$

For HHT the same method gives 8. It is worth understanding why these states work for *every* word, rather than solving a new system from scratch each time.

§1.2 The useful part of a failed match

Take a word w of length m. After observing a finite string s, let

$$q_w(s) = \max\{0 \le j < m : \text{the length-}j \text{ suffix of } s \text{ equals } w_1 \cdots w_j\}, \tag{1.1}$$

provided that w has not already appeared. The integer $q_w(s)$ records the longest suffix of what we have read that could be the beginning of the desired pattern.

The next state is obtained by appending the new letter a and retaining the longest suffix which is a prefix of w. Write this transition as $\delta(j, a)$. If the appended string completes w, we enter a separate absorbing success state †. Otherwise,

$$q_w(sa) = \delta(q_w(s), a).$$

Why does the entire past reduce to a single integer? Every shorter candidate suffix that could become a prefix match after the next letter is itself a suffix of the current longest matching prefix. We can compute its fallback from that prefix alone.

This is the idea behind the failure links in the Knuth–Morris–Pratt algorithm. With a zero-based prefix-function convention,

$$\pi[i] = \text{length of the longest proper border of } w_1 \cdots w_{i+1},$$

where a *border* is simultaneously a prefix and a suffix. In particular,

$$\pi[0] = 0, \qquad \pi_{\mathrm{HHH}} = (0, 1, 2), \qquad \pi_{\mathrm{HTH}} = (0, 0, 1).$$

There is no nonempty *proper* border of a one-letter word. That explains the initial zero; it says nothing about whether arrays are indexed from zero or one.

An instructive check is the string HHT while looking for HTH. The first two heads leave state 1, and the following tail leaves state 2. The earlier failed attempt to match HT did not destroy the opportunity to start matching at the *second* head. For a general pattern, the prefix function organizes precisely these fallback possibilities.

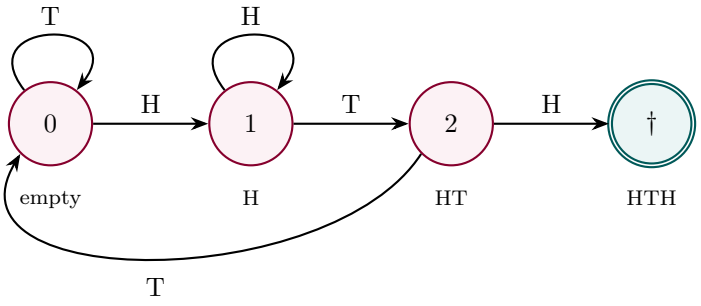


Figure 1.1: The prefix automaton for the first occurrence of HTH. Each arrow consumes one toss. Under a fair coin, each of the two possible outgoing letters has probability 1/2; † stops the process.

§1.3 A finite automaton becomes a Markov chain

The state transition depends only on the current state and on the next independently sampled letter. Thus a random stream of letters turns our deterministic matching automaton into a finite-state Markov chain, stopped on entering †.

For the three transient states $0, 1, 2$, list the probabilities of transitions *between transient states* in a matrix Q. In the order $0, 1, 2$, the two patterns give

$$Q_{\mathrm{HHH}} = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} \\ \frac{1}{2} & 0 & 0 \end{pmatrix}, \qquad Q_{\mathrm{HTH}} = \begin{pmatrix} \frac{1}{2} & \frac{1}{2} & 0 \\ 0 & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & 0 & 0 \end{pmatrix}. \tag{1.2}$$

Rows corresponding to a state from which success is possible sum to less than 1, because we have removed transitions to †. This is why Q is *substochastic*.

Let $\mathbf{1} = (1, 1, 1)^\top$ and let $\mathbf{e}_0 = (1, 0, 0)^\top$. If we start in state 0, the probability of *not* seeing the pattern during the first n tosses is exactly

$$\Pr(T_w > n) = \mathbf{e}_0^\top Q^n \mathbf{1}. \tag{1.3}$$

This identity is already much more informative than the mean. For example,

$$\Pr(T_{\mathrm{HHH}} = 3) = \Pr(T_{\mathrm{HTH}} = 3) = \frac{1}{8},$$

whereas

$$\Pr(T_{\mathrm{HHH}} = 4) = \frac{1}{16}, \qquad \Pr(T_{\mathrm{HTH}} = 4) = \frac{1}{8}.$$

At toss four, the two waiting-time distributions have visibly diverged.

Why can we invert $I - Q$? For a word w of length m, every disjoint block of m tosses has probability 2^{-m} of equaling w. If any such block equals w, then w must already have appeared by the end of that block. The blocks are independent, so

$$\Pr(T_w > km) \le (1 - 2^{-m})^k. \tag{1.4}$$

In particular, $T_w < \infty$ almost surely and $\mathbb{E}T_w < \infty$. The same bound applies from any transient state, ensuring that the powers Q^n tend to zero. Consequently,

$$N = (I - Q)^{-1} = \sum_{n \ge 0} Q^n$$

exists. Its entry N_{ij} is the expected number of visits to transient state j, starting from state i, before absorption. Summing over the possible states gives

$$\mathbf{t} = N\mathbf{1}, \qquad \mathbf{t} = \mathbf{1} + Q\mathbf{t}, \qquad \mathbb{E}T_w = \mathbf{e}_0^\top (I - Q)^{-1}\mathbf{1}. \tag{1.5}$$

Our earlier recurrences were just these matrix equations written componentwise.

This is an algorithm for any fixed pattern and any finite alphabet with positive letter probabilities. There is, however, a much shorter expression for the answer if the letters are independent and equally likely. To find it, we should look at *how a completed word can overlap itself.*

§1.4 How a word overlaps itself

A *border* of a word $w = w_1 \cdots w_m$ is a prefix which is also a suffix. In this section we allow the whole word itself as a border; a *proper* border has length less than m. Write

$$\mathcal{B}(w) = \{k \in \{1, \ldots, m\} : w_1 \cdots w_k = w_{m-k+1} \cdots w_m\}.$$

An overlap can also be described by the shift between two copies of w. Define

$$\mathcal{D}(w) = \{d \in \{0, \ldots, m-1\} : w_{d+1} \cdots w_m = w_1 \cdots w_{m-d}\}.$$

The two sets are equivalent: $d \in \mathcal{D}(w)$ precisely when $m - d \in \mathcal{B}(w)$. In particular, $0 \in \mathcal{D}(w)$, corresponding to the full-length border.

For our three examples,

w	$\mathcal{B}(w)$	$\mathcal{D}(w)$
HHH	$\{1, 2, 3\}$	$\{0, 1, 2\}$
HTH	$\{1, 3\}$	$\{0, 2\}$
HHT	$\{3\}$	$\{0\}$

The first two heads of HHH can also be the last two heads of a preceding copy. For HTH, only one head can be shared between copies at distinct starting positions. The word HHT has no nonempty proper border at all.

Package the self-overlaps into the polynomial

$$R_w(z) = \sum_{d \in \mathcal{D}(w)} 2^{-d} z^d. \tag{1.6}$$

The exponent records the shift, while 2^{-d} records the probability of obtaining the d fresh letters required to finish a second copy. Thus

$$R_{\mathrm{HHH}}(z) = 1 + \frac{z}{2} + \frac{z^2}{4}, \quad R_{\mathrm{HTH}}(z) = 1 + \frac{z^2}{4}, \quad R_{\mathrm{HHT}}(z) = 1.$$

A polynomial extracted solely from the letters of a word will turn out to determine its entire first-occurrence distribution.

§1.5 One identity for the entire distribution

Write

$$f_n = \Pr(T_w = n), \qquad G_w(z) = \sum_{n \geq m} f_n z^n = \mathbb{E}[z^{T_w}], \qquad 0 \leq z < 1.$$

The event that w *ends* at time n has probability 2^{-m} for every $n \geq m$, irrespective of whether w appeared earlier. We can partition this event according to the first occurrence time $T_w = t \leq n$.

Suppose $T_w = t$, and put $d = n - t$. If $d < m$, an occurrence ending at n overlaps the completed word ending at t. It is possible only when $d \in \mathcal{D}(w)$; in that case the d new tosses must reproduce a specified sequence, giving probability 2^{-d}. If $d \geq m$, the m letters in the later occurrence are entirely new and have probability 2^{-m}. Accordingly define

$$c_d = \begin{cases} 2^{-d}, & 0 \leq d < m, \quad d \in \mathcal{D}(w), \\ 0, & 0 \leq d < m, \quad d \notin \mathcal{D}(w), \\ 2^{-m}, & d \geq m. \end{cases}$$

Independence of future tosses, conditional on $T_w = t$, now gives the *renewal identity*

$$2^{-m} = \sum_{t=m}^{n} f_t \, c_{n-t}, \qquad n \geq m. \tag{1.7}$$

On the left we count every occurrence ending at n. On the right we first locate the earliest occurrence, then check how a later copy can follow it. This is where the word's overlaps enter the calculation.

The generating function of the left side is

$$B(z) = \sum_{n \geq m} 2^{-m} z^n = \frac{2^{-m} z^m}{1 - z}.$$

For the coefficients c_d,

$$C(z) = \sum_{d \geq 0} c_d z^d = R_w(z) + \frac{2^{-m} z^m}{1 - z}.$$

Equation (1.7) is precisely the coefficient identity $B(z) = G_w(z)C(z)$. Solving for G_w yields:

Theorem 1.5.1 (First-occurrence generating function)

For any nonempty binary word w of length m in an independent sequence of fair coin tosses,

$$\boxed{G_w(z) = \frac{2^{-m} z^m}{(1 - z)R_w(z) + 2^{-m} z^m}.} \tag{1.8}$$

The polynomial R_w is determined by the self-overlaps of w as in (1.6).

The formula looks rather generous: from a few overlap shifts, it produces every number $\Pr(T_w = n)$, by expanding a rational function as a power series.

The geometric bound (1.4) gives exponential tails, so we may differentiate G_w at $z = 1$ to compute $\mathbb{E}T_w$. Since

$$G_w(1) = 1, \qquad \frac{d}{dz} \log\bigl((1 - z)R_w(z) + 2^{-m} z^m\bigr)\Big|_{z=1} = m - 2^m R_w(1),$$

we obtain

$$\boxed{\mathbb{E}T_w = G_w'(1) = 2^m R_w(1) = \sum_{k \in \mathcal{B}(w)} 2^k.} \tag{1.9}$$

Now the three original answers are almost visible without calculation:

$$\mathbb{E}T_{\mathrm{HHH}} = 2 + 4 + 8 = 14, \quad \mathbb{E}T_{\mathrm{HTH}} = 2 + 8 = 10, \quad \mathbb{E}T_{\mathrm{HHT}} = 8.$$

For an alphabet of size q, with letters chosen independently and uniformly, the same argument replaces 2 by q:

$$\mathbb{E}T_w = \sum_{k \in \mathcal{B}(w)} q^k.$$

If the letters are independent with unequal probabilities $p(a) > 0$, the corresponding expression is

$$\mathbb{E}T_w = \sum_{k \in \mathcal{B}(w)} \frac{1}{p(w_1)p(w_2)\cdots p(w_k)}. \tag{1.10}$$

Indeed, when a later copy overlaps in $m - d$ letters, the d fresh letters have probability $\prod_{i=m-d+1}^{m} p(w_i)$, while the entire word has probability $\prod_{i=1}^{m} p(w_i)$. The ratio is exactly the reciprocal probability of the border prefix of length $m - d$.

§1.6 A gambler who bets on prefixes

There is a rather charming explanation of the sum in (1.9). Before each toss, a new gambler arrives and stakes one unit on the first letter of w. A winning bet doubles the gambler's capital, which is then wagered on the second letter of w, and so on. An unsuccessful bet ends that gambler's participation; a gambler who correctly predicts all m letters is paid and leaves.

Every individual bet is fair. Each arrival contributes one unit, so after n tosses the casino's net wealth is

$$W_n = n - \text{total capital still held by winning gamblers.}$$

Treating a completed winner's payout as part of this total at the stopping instant, the process W_n, up to the first occurrence of w, is a martingale.

When the first complete w arrives at time T_w, which gamblers can still be holding capital? Exactly those who have correctly predicted a suffix of the observed sequence which is also a prefix of w. Since the last m symbols equal w, their successful run lengths are the elements of $\mathcal{B}(w)$. A gambler who has made k correct predictions has capital 2^k. Thus

$$W_{T_w} = T_w - \sum_{k \in \mathcal{B}(w)} 2^k.$$

There are at most m live gamblers, with capital at most 2^m each. Together with the exponential waiting-time bound, this justifies stopping the fair-game martingale at T_w by truncation and dominated convergence. Taking expectations gives the same formula:

$$\mathbb{E}T_w = \sum_{k \in \mathcal{B}(w)} 2^k.$$

The betting picture makes the self-overlaps tangible. When HHH finally arrives, the casino pays one gambler 8, another 4, and another 2. When HHT arrives, only the gambler who predicted all three letters survives to collect 8.

§1.7 Avoiding a word

We can now recover the probabilities of *not yet* having seen w. Let

$$S_n = \Pr(T_w > n), \qquad n \geq 0, \quad \text{and} \quad H_w(z) = \sum_{n\geq 0} S_n z^n.$$

For any positive integer-valued T,

$$\sum_{n\geq 0} z^n \mathbf{1}_{\{T>n\}} = 1 + z + \cdots + z^{T-1} = \frac{1-z^T}{1-z}.$$

Take expectations with $T = T_w$ to obtain

$$H_w(z) = \frac{1 - G_w(z)}{1-z} = \frac{R_w(z)}{(1-z)R_w(z) + 2^{-m} z^m}. \tag{1.11}$$

Compare this with (1.3): the automaton computes the survival probabilities by matrix multiplication, while the overlap polynomial packages the same sequence into a rational function.

For HHH,

$$H_{\mathrm{HHH}}(z) = \frac{1 + z/2 + z^2/4}{1 - z/2 - z^2/4 - z^3/8}.$$

Let a_n count length-n binary words with no occurrence of HHH. Since every length-n word is equally likely,

$$\frac{a_n}{2^n} = \Pr(T_{\mathrm{HHH}} > n).$$

Consequently,

$$\sum_{n\geq 0} a_n x^n = H_{\mathrm{HHH}}(2x) = \frac{1 + x + x^2}{1 - x - x^2 - x^3}. \tag{1.12}$$

Comparing coefficients gives

$$a_0 = 1, \qquad a_1 = 2, \qquad a_2 = 4, \qquad a_n = a_{n-1} + a_{n-2} + a_{n-3} \quad (n \geq 3).$$

For $n \geq 3$, we can see this recurrence directly. A binary word avoiding HHH must contain a last tail. It therefore ends with exactly one of the blocks T, TH, or THH. Removing that final block leaves a shorter word avoiding HHH, and conversely each shorter avoiding word can be extended by any one of these blocks. Thus the initial counts are

$$1,\ 2,\ 4,\ 7,\ 13,\ 24,\ 44, \ldots.$$

A waiting-time problem has produced a Tribonacci-type counting problem, using exactly the same forbidden word.

§1.8 The tail remembers the distribution

The expectation itself can be recovered from the survival probabilities:

$$\mathbb{E}T_w = \sum_{n=0}^{\infty} \Pr(T_w > n). \tag{1.13}$$

To prove it, write the elementary identity

$$T_w = \sum_{n=0}^{\infty} \mathbf{1}_{\{T_w > n\}}$$

for each sample outcome, then interchange expectation and the nonnegative sum. It is also the limit $H_w(1^-)$ of (1.11).

A single expectation summarizes the tail in one number. An entire family of *truncated expectations* retains much more. For integers $k \geq 0$, define

$$C_w(k) = \mathbb{E}[(T_w - k)_+], \qquad (x)_+ = \max\{x, 0\}.$$

For each integer $t \geq 0$,

$$(t-k)_+ = \sum_{n=k}^{\infty} \mathbf{1}_{\{t>n\}}.$$

Consequently,

$$C_w(k) = \sum_{n=k}^{\infty} \Pr(T_w > n). \tag{1.14}$$

Subtract successive values:

$$\boxed{C_w(k) - C_w(k+1) = \Pr(T_w > k).} \tag{1.15}$$

A second difference recovers individual probabilities:

$$\boxed{C_w(k) - 2C_w(k+1) + C_w(k+2) = \Pr(T_w = k+1).} \tag{1.16}$$

Thus the sequence $C_w(0), C_w(1), C_w(2), \ldots$ determines the entire law of T_w. It begins with $C_w(0) = \mathbb{E}T_w$, but each new entry reveals another piece of the tail. Its successive slopes are nondecreasing, because the right side of (1.16) is nonnegative: C_w is discretely convex.

There is a continuous version of the same picture. For any nonnegative integrable random variable X and real $a \geq 0$,

$$C_X(a) := \mathbb{E}[(X-a)_+] = \int_a^{\infty} \Pr(X > t)\, dt. \tag{1.17}$$

Indeed, pointwise, $(X-a)_+ = \int_a^{\infty} \mathbf{1}_{\{X>t\}}\, dt$; Tonelli's theorem allows us to exchange the integral and the expectation. Where the survival function is continuous,

$$C_X'(a) = -\Pr(X > a).$$

At an atom a, the right and left derivatives differ:

$$C_{X,+}'(a) = -\Pr(X > a), \qquad C_{X,-}'(a) = -\Pr(X \geq a).$$

The jump in slope therefore gives $\Pr(X = a)$. A convex function built from expected excesses above thresholds encodes the distribution itself.

For our integer-valued hitting times, $a \mapsto C_{T_w}(a)$ is likewise piecewise linear between consecutive integers. Its values at the integers already suffice to recover the law by (1.16). The waiting-time puzzle has led us to a general principle about random variables: a tail integral is a cumulative record of probabilities, and differentiation can read them back.

§1.9 When two patterns race

There is one final surprise. Suppose two players choose different length-three words and we toss the same fair coin until either pattern appears. The first completed pattern wins.

Let Alice choose HHH and Bob choose THH. Alice wins if and only if the first three tosses are all heads. Once a tail has appeared, any later run of three heads must first pass through the consecutive pattern THH: the first two heads after the last tail already complete Bob's word. Since some winning word appears almost surely,

$$\Pr(\text{THH occurs before HHH}) = 1 - \Pr(X_1X_2X_3 = \text{HHH}) = \frac{7}{8}. \tag{1.18}$$

Bob wins seven times out of eight, even though both players chose a word of length three.

The larger family of such races, often called *Penney's game*, exhibits nontransitive preferences among patterns. Analyzing several competing words requires cross-overlaps between *different* patterns, just as our single-pattern analysis required self-overlaps. The same prefix-automaton method extends by making the success state record *which* word has appeared first.

For two competing words A, B of the same length, we can make a prefix automaton whose transient states are the proper prefixes of *either* word. There are now two absorbing states, one for each winner. If Q is the transient transition matrix and r_B is the column of one-step transition probabilities into Bob's winning state, the vector of Bob's eventual winning probabilities satisfies

$$\mathbf{p}_B = Q\mathbf{p}_B + r_B, \qquad \mathbf{p}_B = (I - Q)^{-1} r_B.$$

Each state records a possible partial match of either competitor. The calculation is as finite as the one for expected waiting time.

As a concrete example, set $A = \text{HHT}$ and $B = \text{THH}$. The transient states are

$$\varnothing, \qquad \text{H}, \qquad \text{HH}, \qquad \text{T}, \qquad \text{TH}.$$

Let p_s be the probability that B wins first when we are in state s. A direct check of the next letter gives

$$\begin{aligned} p_\varnothing &= \tfrac{1}{2}(p_\text{H} + p_\text{T}), & p_\text{H} &= \tfrac{1}{2}(p_\text{HH} + p_\text{T}), \\ p_\text{HH} &= \tfrac{1}{2}p_\text{HH} + \tfrac{1}{2}\cdot 0, & p_\text{T} &= \tfrac{1}{2}(p_\text{TH} + p_\text{T}), \\ p_\text{TH} &= \tfrac{1}{2}\cdot 1 + \tfrac{1}{2}p_\text{T}. \end{aligned}$$

The two boundary values 0 and 1 correspond to completing A or B. The equations yield $p_\text{HH} = 0$, $p_\text{T} = p_\text{TH} = 1$, $p_\text{H} = 1/2$, and finally

$$\boxed{\Pr(\text{THH appears before HHT}) = p_\varnothing = \frac{3}{4}.}$$

The same calculation for four choices produces a small cycle:

First choice A	Reply B	$\Pr(B \text{ wins})$
HHT	THH	3/4
THH	TTH	2/3
TTH	HTT	3/4
HTT	HHT	2/3

Each reply is more likely to occur first than the word it answers, yet following the replies brings us back to the starting word. The overlapping suffix of one word and prefix of another now play the role that a border played for a single word.

This cycle says something subtler than its table of odds. The words HHT and THH have *equal individual expected waiting times*,

$$\mathbb{E}T_{\mathrm{HHT}} = \mathbb{E}T_{\mathrm{THH}} = 8,$$

yet THH beats HHT with probability $3/4$ when both compete on the *same* stream of coin tosses. Separate expectations do not determine the probability that one stopping time precedes another; this probability depends on their joint construction.